IEEESTEC – 19th Student Projects Conference, Niš, 2026.
https://doi.org/organizer will add number later
Original scientific paper

Fiducial: Build Any Robust Multidisciplinary Product with AI Agents

Aleksa Zdravković
12th grade high school student
ETŠ "Nikola Tesla" Niš
Niš, Serbia
aleksazdravkovic@proton.me
ORCID: 0009-0004-1535-1839

***Abstract*—AI coding agents can now write firmware, web applications and even circuit-board and enclosure code, which lets one person attempt products that used to need a team. Building such products showed the author where this falls short: each part can be written quickly, but the facts the parts share are copied between them, and nothing keeps the copies in agreement. This paper presents Fiducial, an open-source platform that gives a whole product, software and hardware alike, one structure. Every fact is declared once, every artifact that uses it is generated from that declaration, and checks on every change refuse anything that no longer agrees with it, so AI agents work inside the checks instead of being trusted. When 20 realistic mistakes were injected into two products, the first version of the checks caught 15; each miss became a general rule, and the current version catches all 20.**

***Keywords*—AI agents, single source of truth, code generation, consistency checking, hardware–software co-design, continuous integration**

I. Introduction

Building a product used to take a team: someone for the circuit board, someone for the firmware, others for the app, the website and the enclosure. AI coding agents such as Claude Code [1] and Codex [2] are changing that. An agent can write a firmware driver, a web page, a database migration or a script that models a case in minutes, so one person can now attempt a whole product.

Building such products – first Outreach [3], an off-grid messaging system made of radios, gateways, an app and printed cases, and then the platform described here – showed the author where the difficulty had moved. Writing each part was no longer the slow step; keeping the parts in agreement was. The radio's firmware and the app disagreed on the layout of a message, and a printed case did not fit the board it was made for. Neither part was wrong on its own. The facts they shared had been copied by hand from one tool to another, one copy changed, and the mistake appeared only when the parts met.

Agents do not remove this problem; they speed it up. An agent works inside one codebase and sees only what is in it, so the agent writing the firmware does not know the board changed, and the one writing the web page does not know the firmware did. Its output is also probabilistic: usually right, but never guaranteed. More code, written faster, in more disconnected places, gives an inconsistent product sooner.

What is missing is not a better tool for any one discipline, but one shared and checked context for all of them: a single place where a product's facts live, from which every part is produced, and which refuses a change that makes the parts disagree. This paper presents Fiducial [4], an open-source platform built to provide it, and tests whether its checks catch the mistakes that people and agents actually make.

II. Background

The idea that a fact should be written only once is not new. Model-driven engineering generates code and configuration from a central model, so that each design decision is stated in one place [5], and model-based systems engineering keeps the mechanical, electrical and software views of a product in one model and checks them for inconsistencies [6]. These approaches put a modelling tool at the centre and an engineer at its controls.

Build systems solved a related problem for software. A build system knows which outputs depend on which inputs and rebuilds only what changed; Nix goes further and identifies every result by a fingerprint of its inputs, so that a build can be reproduced exactly and no change goes unnoticed [7]. This works within software, but a circuit board, an enclosure or a translated web page usually lies outside it.

Coding agents, finally, are guided by context files such as AGENTS.md [8], which describe a repository's layout and rules in plain language. They help, but an agent can still misread or ignore them, and nothing checks its work against the rest of the product. A known way to use a generator that can be wrong is to pair it with a checker that cannot: the generator proposes and a sound verifier decides [9]. Fiducial brings the three ideas together for a whole product: facts in plain text that agents can edit, outputs

generated for every discipline, and checks that decide what is accepted.

III. The Approach

Fiducial treats a product as two kinds of thing. Facts are the decisions that define it: which pin a sensor is connected to, what a button says in each language, the brand colour, the size of the board, the layout of a message between a device and its app. Artifacts are everything built from those facts: source code, schematics, circuit boards, translation files, web pages, enclosure models. Drift is what happens when an artifact stops agreeing with a fact, or with another artifact that uses the same fact (Fig. 1a). The approach consists of three rules that together make drift visible and keep it out of the product (Fig. 1b).

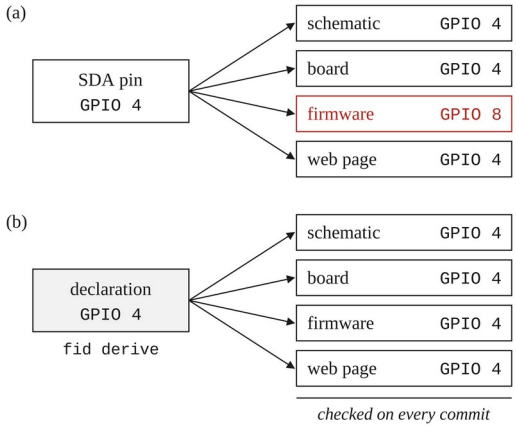


Fig. 1. One fact used in four places: (a) copied by hand, one copy changes and nothing notices; (b) declared once, every copy is generated from the declaration and checked.

Declare once. Every fact is written in exactly one place, a declaration: a typed text file in the product's repository. Typed means the file has a fixed structure that a program can check, not free prose. A fact that exists once has no second copy to disagree with.

Derive everything else. Every artifact that depends on a fact is generated from its declaration by one command, and generated files are never edited by hand. Generation is deterministic: the same declarations always produce exactly the same files, which makes the product reproducible and makes any difference meaningful.

Check every change. Before a change is accepted, checks confirm that every artifact still matches what its declarations produce now, and that the declarations themselves are valid. A failing check names the declaration and the file at fault, so the person or agent knows what to fix.

These rules change the role of an AI agent. The agent still edits declarations and code, but it is no longer trusted to keep anything consistent: generation does that, and the checks confirm it. Its mistakes are not prevented, but they are caught before they reach the product, with a message precise enough for the agent to correct itself. The name comes from electronics manufacturing, where fiducials are the marks on a circuit board that every machine aligns to; the declarations play that role for a whole product.

IV. Implementation

Fiducial is a command-line tool, fid, written in Rust. Fig. 2 shows how a change moves through it.

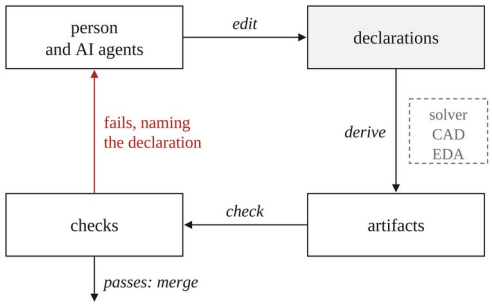


Fig. 2. How a change moves through Fiducial: declarations are edited, generation produces the artifacts with the help of established engines, and the checks either accept the change or return a message naming the declaration at fault.

A. One Structure for Every Product

fid new creates a product repository with a fixed layout, and fid add enables capabilities. A capability is a packaged piece of the platform for one discipline: its declarations, the generator that turns them into artifacts, its checks, and instructions for agents. There are 23, from web applications, translations and legal pages to databases, device messages, firmware and hardware. Because every product shares the same structure, a person or agent who knows one knows them all, and a new product starts from parts that already work and pass their checks. When the platform improves, fid upgrade brings the change into each product while keeping the product's own edits.

B. Declarations and Generation

Declarations are plain text files in version control: the strings of each language, the pages and brand of the website, the colour palette with the contrast each pair of colours must reach, the messages a device exchanges with its app and, for hardware, every part with the net each of its pins connects to. Text was chosen over a graphical modelling tool because coding agents read and write it well, and because every change then goes through ordinary code review. fid derive runs each capability's generator, which reads the declarations and writes the artifacts, from a typed module of translations to a firmware pin map or a KiCad circuit board [10]. A lock file records a cryptographic hash – a short fingerprint of a file's contents – for every input and output, so the platform knows exactly what each artifact was made from.

C. Checks

Checks run in continuous integration, the automatic build that runs on every proposed change, and a change is merged only when they pass. There are three kinds. Freshness: fid derive --check compares every recorded fingerprint with the repository and fails if a declaration changed without its artifacts being regenerated, or if a generated file was edited by hand. Rules: generation refuses declarations that break a rule, such as a language missing a string, two colours with too little contrast, parts that cost more than the declared limit, or a board that does not fit its case. Consumers: compilers and type checkers read the generated files, so code that still uses a renamed fact stops compiling.

D. Engines for Hard Problems

Some generation steps are real engineering problems: where to place the parts on a board, how to connect them, how to build a case around them. The first version placed parts with a search written by hand, and every new product added special rules until the rules began to interact. It was replaced by a different structure: the platform writes the problem down in a fixed format and hands it to an established engine. A constraint solver, CP-SAT [11], receives the rules a placement must satisfy and finds one that does, or proves that none exists; build123d on the OpenCascade kernel [12] builds the case; and KiCad routes the board and checks it against its design rules. Every placement rule is one of eight general kinds, such as inside, apart or near, so a new product needs new declarations rather than new code. When declarations contradict each other, the solver names the smallest set that cannot hold together.

E. Agents in the Loop

Every product carries an AGENTS.md file and one instruction file per capability, so any agent starts with the same description of the product and its rules, and a hook stops an agent from editing a generated file. Agents are asked to fix a problem where the fact lives: a wrong pin in the declaration, a wrong generator in the platform, never in the output. The platform grows the same way. When a mistake gets past the checks, it is not patched for one product but turned into a general rule that protects every product.

V. Example: A Sensor Stick

To show the whole approach on something anyone can inspect, the repository contains a complete example product: a USB-C stick that measures temperature and humidity and shows them on a web page (Fig. 3). It is small on purpose, but it has every discipline a product has: a printed case, a circuit board, firmware and a web page.

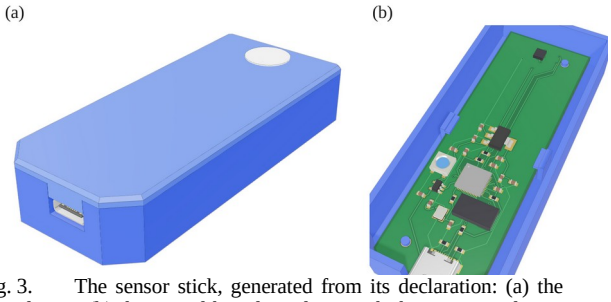


Fig. 3. The sensor stick, generated from its declaration: (a) the closed case; (b) the routed board inside it, with the sensor under its vent.

The person writes four things: the product declaration with its parts and connections, an outline of its shape, the firmware's logic and the page. Everything else is generated: the placed and routed board with its schematic and bill of materials, the case built around the board with an opening for the connector and a vent over the sensor, the files a manufacturer needs, and the code the firmware and the page both use to exchange readings.

Moving the sensor to another pin is one changed line in the declaration. Until the product is regenerated, the check fails; afterwards the board, the schematic, the firmware's pin map and the page's types have all followed, and neither the firmware nor the page needed an edit. If the new pin is one the chip's sensor bus cannot use, the firmware stops compiling – before any board is made.

VI. Evaluation

A. Development

Fiducial was built over six weeks by one person working with AI agents, through 68 reviewed pull requests, and used on two products: the sensor stick and a private product. The sensor stick's board passes its design-rule check with no errors, and moving one pin regenerates 8 files in 37 s.

B. Injected Mistakes

Checks are only useful if they catch the mistakes that people and agents actually make. To measure this, the test borrows the idea of mutation testing [13]: a realistic mistake is introduced on purpose, the checks are run, and the result shows whether they noticed. Twenty mistakes were chosen to cover eight kinds of drift met while building Fiducial and its products (Table I): ten in a fresh web product in English and Serbian, and ten in the sensor stick's hardware, firmware and the interface between them.

Each mistake was applied to a clean copy of the product, the checks ran in the same order as in continuous integration, and the copy was reset. Three correct changes and two runs with no change at all tested the opposite direction: that the checks do not fail without a reason. The test is a script in the repository and runs with one command.

TABLE I. INJECTED MISTAKES, BY KIND OF DRIFT

Kind of drift	**Example**	**0.9.4**	**0.9.11**
Stale output	Domain edited, not re-derived	1/3	3/3
Generated file edited	Pin map edited by hand	5/5	5/5
Missing declaration	String in English only	3/3	3/3
Rule broken	Contrast below 4.5 : 1	3/3	3/3
Declarations conflict	Case too narrow for board	2/2	2/2
Code out of date	Net renamed, old name used	1/1	1/1
Declaration bypassed	Pin typed as a number	0/2	2/2
Not translated	Serbian left in English	0/1	1/1
Total		15/20	20/20

On release 0.9.4 the checks caught 15 of the 20. The five misses were the most useful result, because each was a fact the platform did not yet own. Two were declarations edited without regenerating: only the hardware generator recorded which files it read, so the freshness check did not notice that the others' inputs had changed. In two, the firmware wrote a pin number or a sensor address directly instead of taking it from the declaration. The last was a Serbian string left in English, complete in structure but not translated. Each became a general rule: every generator now records its inputs, firmware that writes a declared value directly is refused, and a translation identical to the default language fails unless it is declared the same on purpose. On release 0.9.11, all 20 are caught – 10 during generation, 9 by the freshness check and 1 by the compiler – and every correct change passes. Fig. 4 shows two of the messages.

(a)

```
$ fid derive
fid-i18n: the catalogs are not complete.
messages/sr.json is missing 1 key(s):
action.delete
```

(b)

```
$ fid derive
firmware types what the board already declares (a pin,
an address, a command or a wait):
firmware/rp2040/src/main.rs:84: takes PIN_4 directly —
take it through `board::<net>!(p)` …
```

Fig. 4. Two failures as the person or agent sees them: (a) a string added in English only; (b) firmware naming a pin directly instead of taking it from the declaration.

The mistakes were chosen by the author of the platform, so they may favour what it was built to catch. Only two products have been built, and neither has yet been manufactured and tested. The test measures the checks, not whether agents working with Fiducial make fewer mistakes than agents working without it.

VII. CONCLUSION

AI agents have made writing each part of a product fast, and that has moved the hard problem to keeping the parts in agreement. Better models will move the line between what a person checks and what is left to agents, but a probabilistic system cannot guarantee its output, and even a deterministic tool is only right about what it was told. Fiducial answers this at the level of the product rather than the codebase: each fact is declared once, every artifact is generated from it, every change is checked, and hard problems go to proven engines. In practice, this let one person working with AI agents keep a product's firmware, web page, circuit board and case in agreement, with every injected mistake caught early by a message that names its cause. Next, the sensor stick will be manufactured and tested, and agents will be compared working with and without the platform on the same tasks. Fiducial and its example are open source [4].

VIII. REFERENCES

[1] Claude Code documentation. Available at: https://docs.anthropic.com/en/docs/claude-code

[2] OpenAI Codex. Available at: https://openai.com/codex/

[3] A. Zdravković, Đ. Jovanović and P. Šubarević, "Outreach: Modular AI-enhanced off-grid mesh communication system for remote environments," IEEESTEC – 18th Student Projects Conference, Niš, 2025.

[4] A. Zdravković, Fiducial, release 0.9.11, 2026. Available at: https://github.com/AleksaZCodes/fiducial

[5] D. C. Schmidt, "Guest editor's introduction: Model-driven engineering," Computer, vol. 39, no. 2, pp. 25–31, 2006.

[6] S. J. I. Herzig, A. Qamar and C. J. J. Paredis, "An approach to identifying inconsistencies in model-based systems engineering," Procedia Computer Science, vol. 28, pp. 354–362, 2014.

[7] E. Dolstra, M. de Jonge and E. Visser, "Nix: A safe and policy-free system for software deployment," in Proc. LISA, 2004, pp. 79–92.

[8] AGENTS.md, a format for guiding coding agents. Available at: https://agents.md/

[9] S. Kambhampati et al., "Position: LLMs can't plan, but can help planning in LLM-Modulo frameworks," in Proc. ICML, PMLR 235, 2024.

[10] KiCad EDA. Available at: https://www.kicad.org/

[11] L. Perron and F. Didier, CP-SAT, Google OR-Tools. Available at: https://developers.google.com/optimization/cp/cp_solver

[12] build123d, a Python CAD library on the OpenCascade kernel. Available at: https://github.com/gumyr/build123d

[13] R. A. DeMillo, R. J. Lipton and F. G. Sayward, "Hints on test data selection: Help for the practicing programmer," Computer, vol. 11, no. 4, pp. 34–41, 1978.